

श्री हित राधावल्लभ लाल जु की जय

NAAM JAP SADHANA

नाम स्मरण

Personal Devotional Sadhana Companion

Product Requirements Document + Technical Architecture

Version 2.0 — All decisions locked

Document Type	PRD + Architecture + Design System + Data Flow
Version	2.0 — Confirmed decisions incorporated
Sampraday	Radhavallabh — Shri Hit Harivansh Mahaprabhu ji
Bhav	Sahachari / Tat-Sukh Bhav
Platform	Android only — APK via Capacitor (locked)
Scope	Single devotee, personal use, offline-first, no login
Default Theme	Sharad Moon (deep indigo, pink lotus)
Starting Target	21,600 naam jap per day
Day Boundary	3:00 AM local time
Mala Style	Tulsi mala — bead slide-down animation per tap
Day-End Trigger	Both: auto on app open after boundary + manual 'End Day'
Backup	Both: auto nightly + manual trigger
Quotes	Fixed library + user-addable custom quotes
Status	Ready for Layer 1 implementation

This is the single source of truth for all implementation decisions. Every AI agent, every session, every line of code aligns with this document. When in doubt — return here.

1. Product Overview

1.1 Purpose

Naam Jap Sadhana (नाम स्मरण) is a personal devotional Android application for a single rasik ananya devotee of the Radhavallabh sampraday of Shri Hit Harivansh Mahaprabhu ji. It serves as a daily sadhana companion for tracking naam jap, reading Shri Hit Chaturasi Ji, and completing the ashtayam seva checklist.

The app is an act of seva — every pixel, every animation, every design decision offered for the happiness of Shri PriyaPriyatam, not for the developer's satisfaction.

1.2 Sampraday Context

Acharya	Shri Hit Harivansh Chandra Mahaprabhu ju (Vanshi Avatar)
Isht Dev	Shri Radhavallabh Lal ju
Swamini	Shri Radha Maharani — Radha Charan Pradhan (her lotus feet are supreme)
Gurudev	Shri Hit Govind Sharan Premanand ji Maharaj
Bhav	Sahachari / Tat-Sukh Bhav — selfless love for the happiness of PriyaPriyatam
Scripture	Shri Hit Chaturasi Ji — 84 pads by Shri Hit Harivansh Mahaprabhu

1.3 Confirmed Platform Decisions

Decision	Value	Rationale
Platform	Android only (APK)	Personal use only. iOS not needed. Capacitor wraps web app cleanly into APK.
Mala Tracker	Tulsi mala bead animation	Each tap = one bead slides down. Feels like holding a real mala.
Day-End Trigger	Both auto + manual	Auto on app open after 3 AM. Manual 'End Day' button always available.
Backup	Both auto + manual	Auto nightly after day-end. Manual trigger in Settings anytime.
Quotes	Fixed + custom addable	Chaturasi Ji + Sevak Vaani + Bhori Sakhi (fixed) + user can add own quotes.
Language	Hindi primary	Devanagari for all devotional text. English for technical labels only.

2. Naam Jap — Core Feature

2.1 Dynamic Target Formula

The mathematical heart of the application. This formula is locked. Never change it without explicit instruction from the project owner.

Condition	Formula
Target met or exceeded $D(n) \geq T(n)$	$T(n+1) = D(n) + 1000$
Target missed $D(n) < T(n)$	$T(n+1) = 2 \times T(n) - D(n)$

Starting target: 21,600 naam jap/day. No cap. No smoothing. Exact raw formula. Increment = 1,000.

Verification Table

Day	Target $T(n)$	Did $D(n)$	Diff	Status	Next $T(n+1)$
1	21,600	25,000	+3,400	Exceeded ✓	26,000
2	26,000	26,000	0	Exact ✓	27,000
3	27,000	20,000	-7,000	Deficit ✗	34,000
4	34,000	0	-34,000	Missed ✗	68,000
5	68,000	0	-68,000	Missed ✗	1,36,000
6	1,36,000	1,50,000	+14,000	Exceeded ✓	1,51,000

2.2 Day Boundary and Edge Cases

- Spiritual day starts at 3:00 AM local. Before 3 AM = still yesterday.
- Missed day: no entry = $D = 0 \rightarrow$ target doubles next day.
- Past edit: triggers full recomputation of ALL subsequent targets and streaks.
- Recomputation is atomic — all records update or none do (IndexedDB transaction).
- Streak continues only if $D(n) \geq T(n)$. Any miss resets streak to zero.

2.3 Input Modes Overview

Mode	Description	Detail
Quick-Add	+108, +1000, +5000 buttons	One tap = instant add. Haptic. Scale press animation.

Custom Number	Direct numeric entry	Numeric keyboard. Any positive integer. Confirm adds to total.
Tap-per-Jap	Large touch area, tap = +1	Giant button. Haptic + golden particle burst per tap.
Tulsi Mala	Bead slide animation (see §2.4)	The most immersive mode. Full spec in Section 2.4.

All 4 modes feed a single $D(n)$ counter. Running total always shows: total jap + mala count + remainder. Example: 25,000 = 231 माला + 52.

2.4 Tulsi Mala Tracker — Complete Specification

This is the most sacred and most carefully designed feature. It must feel like holding a real tulsi mala in your hands. Every technical decision serves that feeling.

2.4.1 Visual Design — The Mala

The mala is displayed as a vertical close-up view — as if the devotee is holding the mala between their hands and it hangs down in front of them. The camera angle is extreme close-up, filling most of the screen, so each tulsi bead is large and tactile.

Bead Appearance

Bead material	Tulsi (holy basil) wood — dark warm brown, natural grain texture visible
Bead shape	Slightly oval/oblate — not perfect spheres. Real tulsi beads are not perfectly round.
Bead size	60–70dp diameter on screen. Large enough to feel physical.
Bead texture	3D rendered or high-quality photo texture. Wood grain lines. Small natural imperfections.
Bead colour	Dark reddish-brown (#4A2010 to #6B3018). Warm. Aged. Sacred.
Bead highlight	Soft specular highlight on top-left suggesting the overhead light of a pooja room.
Thread / sutra	Thin saffron/red thread visible between beads. The sutra that holds the mala.
Thread colour	Deep saffron (#CC5500) or deep red (#8B1A00). User can choose in settings.
Sumeru bead	Slightly larger (80dp), different texture — the sumeru is the anchor bead. Gold marker or different wood colour.
Count display	Overlaid below the mala: '42 / 108' and 'माला: 3' — understated, never crowding the mala visual.

Layout — Portrait Screen

- Top 15% — header: 'तुलसी माला' label + current total count
- Middle 65% — the mala itself. This is the primary visual. It dominates the screen.
- Bottom 20% — mala count + bead count + undo button + done button

The Mala in Resting State

108 beads arranged vertically in a single column, hanging from top to bottom. The topmost bead is just below the header. The Sumeru bead is at the top, distinguished. Below it: 108 jap

beads. The beads that have been counted are slightly lower on screen than uncounted beads — as if they have been slid along the thread.

2.4.2 The Tap Animation — Core Interaction

This animation is the heart of the entire mala feature. It must feel physical. The devotee taps — one bead slides down — exactly like sliding a bead along a real mala thread. This feeling must be right.

Animation Sequence — Per Tap

Step	Timing	Visual	Feel
1	0ms	User finger touches the next uncounted bead.	—
2	0–40ms	Bead compresses slightly (scale 1.0 → 0.92). Contact confirmation.	Haptic: light impact (IMPACT_LIGHT)
3	40–160ms	Bead slides downward along thread. Cubic-bezier(0.25, 0.46, 0.45, 0.94) — fast start, gentle ease-out. Travels 72–80dp downward.	Haptic: medium click at end of slide (IMPACT_MEDIUM)
4	160–220ms	Bead settles into counted position with micro-bounce (scale 1.0 → 1.04 → 1.0). The thread visually relaxes.	Audio: optional soft 'tak' sound (wood on wood)
5	220ms+	Bead is now in counted position. Subtle golden rim glow fades in over 400ms — counted beads have a faint warm highlight vs uncounted.	—
6	Any time	Uncounted beads shift up slightly to fill the gap — the mala contracts upward naturally as counted beads accumulate below.	Thread physics: organic, not rigid

Animation Technical Spec

Framework	React Native Reanimated 3 or CSS animations (depending on Three.js integration approach)
Bead slide curve	cubic-bezier(0.25, 0.46, 0.45, 0.94) — ease-out, natural gravity feel
Slide distance	72–80dp downward on the thread axis
Slide duration	120ms total travel time
Compression	scale(0.92) on touch-down, returns to 1.0 at start of slide
Settle bounce	scale 1.0 → 1.04 → 1.0, duration 60ms, ease-in-out

Counted glow	box-shadow / rim light: rgba(255, 200, 80, 0.25), fade-in 400ms
Haptic sequence	IMPACT_LIGHT on touch, IMPACT_MEDIUM on settle
Thread animation	Bezier curve connecting beads updates fluidly as beads move
Max simultaneous	One bead slides at a time. Tap during animation: queued, not dropped.

2.4.3 The Sumeru Bead — Mala Completion

When bead 108 slides down, the mala is complete. The Sumeru bead is reached. This moment must feel like arriving — not just a number incrementing.

Sumeru Completion Sequence

- Bead 108 slides into place — normal animation.
- 0.2 second pause — the mala is still.
- The Sumeru bead pulses with warm golden light — 3 gentle pulses, 600ms each.
- All 108 counted beads briefly glow amber-gold simultaneously — like a wave of light running down the mala from sumeru to last bead.
- Counter increments: 'माला: 3' → 'माला: 4'. The number rises with a gentle upward float animation.
- Golden particle burst — 20 particles from the sumeru bead position, brief, radial.
- Haptic: NOTIFICATION_SUCCESS pattern (not just a click — a completion feel).
- Optional audio: a soft bell tone (user configurable).
- Mala resets: all 108 beads animate back to uncounted position over 800ms — smooth, flowing, like lifting the mala to start again.
- D(n) increments by 108. Zustand + IndexedDB write.

The mala reset animation (beads returning to top) must feel like lifting the mala, not like a progress bar resetting. It should take a full 800ms and feel like the end of a mala — not a UI state change.

2.4.4 Long-Press to Remove Last Bead (Undo)

Gesture	Long-press on the last counted bead (bottommost counted bead)
Threshold	500ms hold
Visual cue	After 300ms hold: bead begins to glow red-orange (warning), scales up slightly
Trigger	At 500ms: bead slides back up to uncounted position. Same animation but reversed and slightly slower (180ms).
Haptic	IMPACT_HEAVY at trigger
Limit	Can undo one bead at a time. Cannot undo across mala completion.
Counter	D(n) decrements by 1.

2.4.5 3D Rendering Approach

The tulsī beads are rendered as Three.js 3D objects — not flat circles or CSS elements. This is what gives them their physical, tactile quality.

Three.js Bead Implementation

```
// Each bead: SphereGeometry with slight Y-scale compression
const beadGeo = new THREE.SphereGeometry(1, 32, 24);
beadGeo.scale(1, 0.88, 1); // Slightly oblate – real tulsī shape

// Tulsī wood material
const beadMat = new THREE.MeshStandardMaterial({
  map: tulsīWoodTexture,          // 512x512 wood grain texture
  roughness: 0.85,                // Matte wood surface
  metalness: 0.02,                // Almost none – it is wood
  normalMap: tulsīNormalMap,      // Subtle surface detail
  color: new THREE.Color('#5A2D14')
});

// Thread: TubeGeometry following a bezier curve
// Updates dynamically as bead positions change
const threadCurve = new THREE.CatmullRomCurve3(beadPositions);
const threadGeo = new THREE.TubeGeometry(threadCurve, 108, 0.04, 8, false);
```

Camera for Mala Mode

Position	Camera is very close to the mala — extreme close-up
FOV	35–40 degrees (narrower than main app FOV of 60 — this feels more intimate)
Movement	Subtle gyroscope-driven parallax — the mala breathes slightly in 3D space
Touch	Touch on individual beads detected via Three.js raycasting
Depth of field	Beads in focus. Background (the animated Vrindavan frame) softly blurred.
Lighting	Warm single directional light from above-right (pooja room overhead lamp feel). Ambient fill from below.

2.4.6 Texture Assets Required

Asset	Size	Description
tulsī_wood_diffuse.jpg	512×512	Dark reddish-brown wood grain. Real tulsī reference.
tulsī_wood_normal.jpg	512×512	Normal map for surface micro-detail — pores, grain.

tulsi_wood_roughness.jpg	512×512	Roughness map — slightly varied across grain lines.
sumeru_diffuse.jpg	512×512	Sumeru bead — same base but different colour/markings.
thread_diffuse.jpg	256×64	Saffron/red thread texture — woven fibre detail.

Texture generation prompt for AI image tools: 'Top-down extreme macro photograph of tulsi (holy basil) wood. Dark reddish-brown, aged, polished surface with natural wood grain lines and tiny pores visible. Warm amber studio lighting. No white specular hotspots. 512x512 seamlessly tileable texture.'

2.4.7 Performance Requirements

Target frame rate	60fps during bead slide animation on mid-range Android (Snapdragon 680+)
Bead geometry	LOD: near beads = 32×24 sphere, far beads = 16×12. Thread visible beads only.
Texture compression	ETC2 format for Android WebGL. Pre-compressed at build time.
Animation budget	Bead slide: single bead transform per tap. No physics simulation — keyframe only.
Memory	108 bead meshes share ONE geometry + ONE material via instancing (InstancedMesh).
Load time	Mala mode must be ready within 300ms of entering the tab.

InstancedMesh Approach

```
// 108 beads via InstancedMesh — single draw call for all beads
const instancedMala = new THREE.InstancedMesh(beadGeo, beadMat, 108);

// Each bead gets its own matrix (position + rotation + scale)
for (let i = 0; i < 108; i++) {
  const matrix = new THREE.Matrix4();
  matrix.setPosition(0, startY - i * beadSpacing, 0);
  instancedMala.setMatrixAt(i, matrix);
}

// On tap: animate only the target bead's matrix
// 120ms slide → update matrix per frame via requestAnimationFrame
```

3. Remaining Feature Specifications

3.1 The 6 Seva Sections

#	Section	Core Functionality
1	नाम जप	Primary. 4 input modes including Tulsi Mala. D(n) counter. Progress ring.
2	श्री हित चतुरासी जी	84 pads. Track daily reading pad. Checkbox per pad. Progress 0/84. Pad text display.
3	मानसिक सेवा	Ashtayam seva checklist (Shringar, Raj Bhog, Sandhya Aarti, Shayan). Notes journal. Streak.
4	प्रार्थना	Daily prayer toggle. Optional prarthana text display. 7-day completion log.
5	भक्त माल	Chapter tracker. Reading progress (page N/total). Notes. Reading streak.
6	चरणामृत	Simple yes/no daily. Monthly calendar grid showing dots. Monthly completion stats.

3.2 Day-End Analysis

Trigger: BOTH automatic (app open after 3 AM, if yesterday had an entry) AND manual ('End Day' button always visible on dashboard).

Target vs completed	Animated counter showing T(n) and D(n) side by side
Difference	Surplus (+) in green, deficit (–) in amber. Exact number.
Tomorrow's target	T(n+1) calculated live. Shows the formula result with explanation.
Streak	Current streak count. Golden flame animation if streak continues.
All 6 sections	Mini status row — tick or cross for each section today.
Quote card	Random pad — no repeat within 7 days. Source attributed. Refresh button.
Emotional state	Celebratory if D ≥ T. Compassionate (never discouraging) if D < T.
Close button	'राधे राधे 🙏' — closes modal and marks analysis as seen for today.

3.3 Quote System

Source 1 (fixed)	Shri Hit Chaturasi Ji — all 84 pads (primary source)
Source 2 (fixed)	Sevak Vaani
Source 3 (fixed)	Bhori Sakhi ke Pad
Source 4 (user)	Custom — user can add, edit, delete own quotes/pads

Rotation rule	No quote shown again within 7 days (tracked via lastShown timestamp)
Display	Full Devanagari text. Attribution. Pad number if Chaturasi. Refresh icon.

3.4 Calendar — 4 Day States

State	Visual	Condition
Exceeded	Gold glow + ✓✓	$D(n) > T(n)$
Met	Green ✓	$D(n) = T(n)$
Partial	Amber + %	$0 < D(n) < T(n)$
Missed	Grey dim	$D(n) = 0$, past date
Future	Empty	Date not yet reached

3.5 Reminders (5 types, all configurable)

Type	Default	Message Content
Morning prabhat	4:00 AM	Prabhat vandan + today's target + radhey radhey
Evening update	7:00 PM	Current count, remaining, encouragement
Day-end analysis	10:00 PM	Analysis ready — tap to see tomorrow's target
Near target	Dynamic	Triggers when 5,000 jap remain today
Spiritual message	6:00 AM	Random pad/quote from vaani

4. Design System

4.1 Design Philosophy

Sacred luxury glassmorphism. When opened at 3 AM for naam jap, the app should feel like stepping into the Nikunj — intimate, alive, sacred. Not a productivity app. Not a meditation app. Vrindavan.

4.2 Visual Constraints — Absolute Rules

NEVER use in any component, ever:

- Om symbol (ॐ) — wrong sampraday
- Rudraksh — Shaiva, not Radhavallabh
- Garuda — not this sampraday
- Deity forms depicted directly
- box-shadow with spread > 0
- Flat coloured card backgrounds
- border-radius below 10px
- Animation duration above 600ms (exception: mala completion = 800ms)
- Standard Tailwind palette colours
- White backgrounds, hard black text
- The Bootstrap shadow: box-shadow 0 4px 12px rgba(0,0,0,0.1)

Visual vocabulary from Chaturasi Ji only:

- Yamuna river banks — sada basant rahat vrindavan, pulin pavitra subhag yamuna tat
- Madhavi lata, malati, juhi, champak, bakul — from pada 27
- Nikunj bower — pada 7: aaju nikunj manju mein khelat
- Kadamba, tamala trees — dari, peacock, parrot
- Sharad Purnima full moon, lotus pool — pada 26
- Diya flame, petal carpet — shayan scene
- Golden lotus — central symbol of Radha Charan Pradhan

4.3 The 5 Themes

#	Name	Mood	Colours
1	Vrindavan Dawn	Warm, sacred	Rose-gold, pearl, champak ivory. Warm palette.
2 ★	Sharad Moon (DEFAULT)	Lunar, mystical	Deep indigo, pink lotus, moonlight silver.
3	Nikunj	Lush, intimate	Emerald, warm gold, bower green.

4	Van Vihaar	Fresh, alive	Champak gold, forest green, Yamuna silver.
5	Shayan	Still, midnight	Midnight blue, diya amber, cool silver.

4.4 Design Tokens (tokens.css — single source of truth)

Typography

Devanagari UI	Noto Sans Devanagari
Devanagari decorative	Tiro Devanagari Hindi
Numbers	DM Sans Bold
Weights	400 regular and 700 bold ONLY — never 500, never 600
Scale	44px hero / 28px large / 20px medium / 16px body / 14px secondary / 12px tertiary / 10px labels

Radius hierarchy

--radius-card	24px — all glass cards
--radius-button	14px — all buttons
--radius-chip	999px — pills, section tabs
--radius-input	12px — text inputs
--radius-small	10px — small elements, badges

Text opacity — 3 levels ONLY

Primary	rgba(255,255,255, 1.00) — main content, headings
Secondary	rgba(255,255,255, 0.65) — supporting text
Tertiary	rgba(255,255,255, 0.38) — labels, hints, timestamps

Glass card formula — use exactly this everywhere

```
background:      rgba(255,255,255, 0.04)
border:          1px solid rgba(255,255,255, 0.08)
backdrop-filter: blur(20px)
border-radius:   var(--radius-card)
```

5. Technology Stack

5.1 Core Stack (Locked)

Technology	Version	Purpose
React	18.x	Component architecture. Screens, cards, inputs. One file per piece.
Vite	5.x	Build tool. Fast dev server. Optimised production bundle.
Three.js	r160	3D scene: orbiting sacred name, tulsi mala beads, particles, glow, gyro.
@react-three/fiber	8.x	React wrapper for Three.js. Lifecycle management. R3F patterns.
Zustand	4.x	Global state: today's data, theme, active section, settings.
idb	7.x	IndexedDB wrapper. Atomic transactions. Local persistence.
Capacitor	6.x	Android APK packaging. Native plugins bridge. SecureStorage.
CSS Custom Properties	Native	Design system. ALL visual values live here. No Tailwind.

5.2 Capacitor Plugins

@capacitor/local-notifications	5 reminder types — scheduled and dynamic
@capacitor/preferences	OAuth token secure storage (native keychain/keystore)
@capacitor/haptics	Physical tap feedback for mala counter and jap input
@capacitor/device	Gyroscope/accelerometer for 3D parallax
capacitor-google-drive (community)	Google Drive backup and restore

5.3 Google Fonts

Noto Sans Devanagari	All Hindi UI text — clean, legible, respectful
Tiro Devanagari Hindi	Pad display, decorative headings — classical feel
DM Sans	Numbers, counters, statistics — modern, clear

5.4 Why This Stack

React + Vite: every component is isolated. Adding a section or feature means one new file — nothing else changes. No surprises later.

Three.js r160: the only library capable of rendering the tulsi mala at 60fps in a mobile WebGL context with proper texture mapping, InstancedMesh for 108 beads in one draw call, and post-processing bloom for the orbiting sacred name.

CSS Custom Properties over Tailwind: this app's design system is unique. Tailwind's utility classes cannot express 5 theme variants, glassmorphism values, or the specific radius hierarchy without hacks. CSS variables are the right tool.

Capacitor over React Native: the entire visual system is web-based. Capacitor wraps it into a real APK without requiring a native renderer. The Three.js canvas runs in WebView at full GPU access on Android.

6. Data Architecture

6.1 Storage Layer

IndexedDB (idb)	Primary: all DailyRecords, settings, custom quotes, quote rotation history
Zustand	Session: today's running total, UI state, active theme — never persisted across app restarts
Capacitor Preferences	OAuth token only — native secure storage, never accessible via JS
public/frames/	Static JPG frame sequences — served directly, not in IndexedDB
User audio	MP3 placed manually by user. Filename string stored in AppSettings.

6.2 DailyRecord Schema

Field	Type	Description
id (PK)	TEXT	YYYY-MM-DD. Primary key.
date	TEXT	ISO date string. Same as id.
target	INTEGER	T(n) at day start. Never changes mid-day.
did	INTEGER	D(n) — running total, updates on every add.
nextDayTarget	INTEGER	T(n+1) — computed and stored on day close.
streakCount	INTEGER	Streak as of this day. Recomputed on past edits.
checkNaamJap	BOOLEAN	Auto-true when did > 0. Manually untickable.
checkChaturasi	BOOLEAN	Manual toggle.
checkMaansikSeva	BOOLEAN	Manual toggle.
checkPrarthana	BOOLEAN	Manual toggle.
checkBhaktMaal	BOOLEAN	Manual toggle.
checkCharnamrit	BOOLEAN	Manual toggle.
quoteShown	TEXT	Quote UUID shown in day-end analysis.
notes	TEXT	Free text. Max 500 chars.
createdAt	INTEGER	UTC epoch milliseconds.
updatedAt	INTEGER	UTC epoch milliseconds. Updates on every write.

6.3 Data Flow — Happy Path

```
App opens
→ dayBoundary.js : is it a new spiritual day? (hour >= 3 AM)
→ db.js           : load today's DailyRecord (create if new day)
→ useAppStore     : hydrate with today's data
```

→ Dashboard : renders target, did, progress ring, checklist

User taps +108 (Quick-Add)

→ useAppStore : increment did by 108 (instant UI)
→ db.js : atomic IndexedDB write (async)
→ ProgressRing : re-renders with new D/T percentage
→ Haptics : IMPACT_LIGHT
→ if did >= target : streak increments, golden celebration particles

App opens after 3 AM (new spiritual day)

→ dayBoundary.js : previous day detected
→ targetEngine.js : $T(n+1) = D(n)+1000$ or $2*T(n)-D(n)$
→ db.js : write nextDayTarget to yesterday's record
→ db.js : create today's record with computed target
→ quoteEngine.js : select pad not shown in last 7 days
→ DayEndModal : auto-shows if yesterday had an entry

6.4 Data Flow — Past-Day Edit (Chain Recompute)

User taps any past date in Calendar → Day Detail → Edit

→ Confirmation dialog: 'इससे आगे के सभी लक्ष्य पुनः गणना होंगे'
→ User confirms
→ db.js: BEGIN IndexedDB transaction
→ Load all DailyRecords from date k onwards, sorted ascending
→ For i = k to today:
 if did[i] >= target[i]: nextTarget = did[i] + 1000
 else: nextTarget = 2*target[i] - did[i]
 streak[i] = did[i] >= target[i] ? streak[i-1] + 1 : 0
 write updated record[i]
 set target[i+1] = nextTarget
→ transaction.COMMIT() ← all records saved
→ if ANY failure: transaction.ROLLBACK() ← nothing changes
→ Zustand refreshes from db
→ UI re-renders

Data integrity rule: The recomputation chain is ALL-OR-NOTHING. Power loss mid-recompute → IndexedDB rolls back. No partial chain state is ever persisted.

7. Application Architecture

7.1 Component Hierarchy

App	
└ ThemeProvider	CSS variable injection per active theme
└ FramePlayer	Background: 50fps JPG frame sequence
└ ThreeScene	3D canvas: orbit + particles (behind all UI)
└ Router	
└ Splash	
└ Dashboard	
└ SectionNav	Horizontal scrollable section pills
└ NaamJap	ProgressRing + InputModeTab + TulsiMala
└ HitChaturasi	PadCard + PadList + ProgressBar
└ MaansikSeva	SevaChecklist + NotesJournal
└ Prarthana	Toggle + TextDisplay + Log
└ BhaktMaal	ChapterTracker + NotesJournal
└ Charnamrit	Toggle + MonthGrid + Stats
└ Calendar	MonthGrid + DayDetail
└ History	Tabs: Today / Weekly / Monthly / Lifetime
└ Settings	ThemeSelector + AudioSettings + Reminders
└ DayEndModal	Global overlay – auto + manual trigger
└ BottomNav	Fixed: Home / Calendar / History / Settings

7.2 State Management (Zustand)

todaySlice	did, target, streakCount, all 6 section booleans, undo stack
uiSlice	activeSection, activeScreen, isDayEndModalOpen, malaModeActive
settingsSlice	theme, sacredName, audioSettings, reminderSettings, backupSettings
historySlice	Cache of recent 30 DailyRecords for fast chart rendering

Rule: Zustand is UI state. IndexedDB is truth. On app load, Zustand hydrates from IndexedDB. On user action: Zustand updates first (instant UI), IndexedDB writes async. If write fails: Zustand reverts.

7.3 3D Scene Architecture

FramePlayer.jsx	Canvas behind everything. Plays extracted JPG frames at 50fps. Theme selects frame folder.
ThreeScene.jsx	Three.js WebGL canvas. Transparent BG. Sits above FramePlayer, below UI.
OrbitText.js	3 TextGeometry instances at radii 3/5/7 units. +20°/-15°/+10° per second. Emission glow + bloom.

TulsiMala.jsx	108 InstancedMesh beads + thread TubeGeometry. Raycasting for tap detection. Slide animations.
ParticleSystem.js	50 petals (slow drift) + 100 sparkles (random). Theme colours. Burst on mala completion.
GyroInput.js	Device orientation → camera offset (parallax). Touch drag fallback.

7.4 Build Layers (Strict Implementation Order)

#	Layer	Files	Status
1	Scaffold + Design	package.json, vite.config, tokens.css, 5 theme CSS, global.css	Pending
2	Data + Logic	db.js, useAppStore, targetEngine, streakEngine, dayBoundary, numberFormat, quotes, quoteEngine	Pending
3	Background + 3D	FramePlayer, ThreeScene, OrbitText, TulsiMala, ParticleSystem, GyroInput	Pending
4	Core Screens	GlassCard, ProgressRing, BottomNav, SectionNav, Splash, Dashboard, 6 sections	Pending
5	Secondary Screens	Calendar, History, DayEndModal, Settings, ThemeSelector	Pending
6	Platform + APK	Notifications, Audio, Backup, Widget, Capacitor config	Pending

8. AI Agent Workflow

8.1 Agent Assignments

Agent	Role	Assigned Tasks
Claude Opus 4.6	Architect	Layer 1 design decisions, Layer 2 logic (targetEngine esp.), schema design, chain recompute algorithm, complex debugging
Claude Sonnet 4.6	Builder	All React components, CSS files, screens, Three.js integration, TulsiMala component, iterative refinement
Gemini 2.5 Pro High	Reviewer	Cross-layer audit, formula verification, visual QA from screenshots, long-context review
Gemini 2.5 Flash	Grunt	Quote data formatting, 84 pads as JSON, boilerplate, barrel exports, data transforms
ChatGPT OSS 120B	3D Specialist	TulsiMala Three.js geometry, InstancedMesh setup, bead slide animation math, glow shader, particle system

8.2 Session Protocol

- 1. Read NAAM_JAP_AGENT_CONTEXT.md (project root)
- 2. Load ClaudeMem — check current layer and last completed task
- 3. Run /superpowers:brainstorm before ANY new layer or complex feature
- 4. Run /superpowers:write-plan to generate the task list
- 5. Load @frontend-design + @ui-ux-pro-max before any UI component
- 6. Run /superpowers:execute-plan with review checkpoints
- 7. After each layer: Gemini Pro High reviews code + screenshots
- 8. Update ClaudeMem. Commit to git. Then next layer.

8.3 TulsiMala Component — Agent Split

The TulsiMala component requires a specific handoff between ChatGPT (3D math) and Sonnet (React integration). Never ask one model to do both.

ChatGPT task	Write standalone Three.js r160 code: 108 InstancedMesh beads, thread TubeGeometry, raycasting for tap detection, bead slide animation (120ms, cubic-bezier), sumeru completion sequence. No React. Pure Three.js.
Sonnet task	Wrap ChatGPT's Three.js code in a React component. Add R3F lifecycle. Connect to Zustand store. Handle theme prop. Expose onBeadTap and onMalaComplete callbacks. Clean up on unmount.

Gemini task	Review the assembled TulsiMala component for: 60fps performance on mid-range Android, memory leaks, animation correctness against spec in §2.4.
-------------	---

9. Security and Data Integrity

9.1 Threat Model

Risk	Severity	Mitigation
Chain corruption during recompute	CRITICAL	All-or-nothing IndexedDB transaction. Rollback on any failure.
OAuth token exposure	HIGH	Capacitor Preferences (native). Never in JS memory longer than needed.
Dependency CVEs	MEDIUM	npm audit before every APK build. Zero high/critical tolerance.
Formula logic error	HIGH	Superpowers TDD: failing test before implementation. Verified against table in §2.1.2.
Partial mala state	LOW	Bead tap queue. No tap dropped. Zustand reverts on IndexedDB failure.

9.2 Pre-APK Checklist

- [] All DailyRecord writes use IndexedDB transactions with rollback
- [] Chain recomputation is atomic — all or nothing
- [] OAuth token in Capacitor Preferences only
- [] npm audit: zero high or critical vulnerabilities
- [] App works fully offline — no network = full functionality
- [] Formula verified against §2.1.2 verification table
- [] Day boundary tested across 3 AM crossing
- [] Past-day edit tested for chain integrity across 7+ days
- [] TulsiMala at 60fps on Snapdragon 680 (mid-range Android)
- [] All 5 themes render correctly and switch without reload

10. Quick Reference

10.1 The Formula

$T(n+1) = D(n) + 1000$ if $D(n) \geq T(n)$ [met / exceeded]
 $T(n+1) = 2 \times T(n) - D(n)$ if $D(n) < T(n)$ [missed]
Starting target: 21,600
Increment: 1,000
Day boundary: 3:00 AM local
Missed day: $D = 0 \rightarrow$ target doubles

10.2 Indian Number Format

1,000 / 21,600 / 1,00,000 / 9,99,999 / 12,34,567

10.3 Design Rules (3 lines)

Glass: `bg rgba(255,255,255,0.04) + border rgba(255,255,255,0.08) + blur(20px)`
Text opacities: 1.00 / 0.65 / 0.38 – only these three
Radius: 24px cards / 14px buttons / 999px pills / 12px inputs

10.4 Agent Routing

Opus → Architecture + Layer 2 logic
Sonnet → All React components + screens (Layers 3/4/5)
GPT → Three.js 3D math only (TulsiMala, OrbitText, particles)
Gemini → Cross-layer code review + visual QA
Flash → Quote data formatting + boilerplate

10.5 The 6 Sections

1. नाम जप
2. श्री हित चतुरासी जी
3. मानसिक सेवा
4. प्रार्थना
5. भक्त माल
6. चरणामृत

श्री राधावल्लभ लाल जु की जय

श्री हित हरिवंश महाप्रभु की जय

राधे राधे 🙏